Real-world (convert a real function to async) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python, plus how Doc12's real FastAPI service would actually need this). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

You're taking a normal (synchronous) function that calls the OpenAI API, and turning it into an async function that calls the API without blocking.

Right now, the function calls `client.chat.completions.create(...)` and just waits — nothing else can happen while it waits. The async version uses a different client and a different way of waiting, so other work could happen at the same time in a real program.

Things to use:

- `from openai import AsyncOpenAI` instead of `OpenAI`.
- `async def` on the function.
- `await` in front of the API call.
- `asyncio.run(...)` to actually run it from plain code.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

The exercise is really about three Python/OpenAI SDK pieces changing together, while the overall logic of the function stays identical.

The starting point (a simplified Doc04-style function):

```
# async_client_conversion_practice.py
from openai import OpenAI

client = OpenAI()

def ask(question):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content
```

The exact pieces:

- **The client class:** `AsyncOpenAI()` instead of `OpenAI()` — a separate class in the same `openai` package, built so every network-calling method on it returns something you have to `await`.
- **The function signature:** `async def ask(question: str) -> str:` — calling it doesn't run the body immediately, it returns a coroutine object.
- **The `await` keyword:** `response = await client.chat.completions.create(...)` — this is what actually pauses this coroutine (without blocking the whole program) until the API responds.
- **Running it from plain code:** `asyncio.run(ask("..."))` — you can't just call `ask("...")` directly at the top level anymore.

Think about what happens to this function once it's inside Doc12's FastAPI service, handling real user traffic instead of running once in a terminal. Two things change that don't matter in a quick script but matter a lot in production: what happens when the API call hangs, and what happens when the service is shutting down mid-request.

An `AsyncOpenAI()` call with no timeout can, in principle, wait forever if OpenAI's servers stop responding — and unlike a blocking `time.sleep()` mistake, this won't even look wrong from the

outside; the request just never comes back. And an `async def` route that's mid-`await` when the server process gets asked to shut down needs that `await` to actually be cancellable, not stuck.

The real design question isn't just "how do I call the API without blocking" — it's "what should happen if this specific call is unusually slow, or if the whole program needs to stop while this call is still in flight?"

The extra pieces that answer that question:

- **A client-level timeout:** `AsyncOpenAI(timeout=30.0)` — every call made through this client gets a 30-second cap by default, instead of trusting each caller to remember to wrap every single call in its own `asyncio.wait_for(...)`.
- **`asyncio.CancelledError`** — when an `await`ed call gets cancelled (a timeout, a shutting-down server), Python raises this inside the coroutine at the point it was waiting. Code that wraps the call in a bare `except Exception:` will accidentally swallow this too — which is exactly the kind of mistake Doc01 warned about with bare `except:` blocks, just showing up in async form.

Difference between Basic and Intermediate: Basic trusts the API call to always come back in a reasonable time. Intermediate also adds a default timeout at the client level, so a hung call fails loudly and boundedly instead of freezing a request (and, in a real server, the worker handling it) forever — and calls out `asyncio.CancelledError` specifically, since it's the one exception type async code has to let propagate, not swallow.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
import AsyncOpenAI instead of OpenAI

create the client:
    client = AsyncOpenAI()

make a function called ask that takes one thing (question), marked async:
    call client.chat.completions.create, with await in front of it
    same model and messages as before
    return the message content, same as before

run it:
    use asyncio.run(ask("some question")) to actually get an answer,
    since you can't just call ask("some question") directly anymore
```

Here's almost the whole thing — just try running it and reading it line by line:

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI
import asyncio

client = AsyncOpenAI()

async def ask(question):
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content
```

Expected output if you run just this: nothing — add `answer = asyncio.run(ask("What is 2 + 2?"))` and `print(answer)` below it to see the model's reply.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
import:
    from openai import AsyncOpenAI
    import asyncio

create client:
    client = AsyncOpenAI()

define:
    async def ask(question: str) -> str:
        response = await client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": question}],
        )
        return response.choices[0].message.content

run:
    answer = asyncio.run(ask("What is 2 + 2?"))
    print(answer)
```

Notice the function body is line-for-line the same as the sync version, except for `await` in front of the API call. Write the whole thing yourself, then compare against the Solution.

Once that's working, add a client-level timeout and catch the specific error it raises:

```
create the client with a timeout:
    client = AsyncOpenAI(timeout=30.0)

same ask() function as before

wrap the call site to see a slow/hung call fail cleanly:
try:
    answer = asyncio.run(ask("some question"))
    print(answer)
except Exception as e:
    print("the call failed or ran out of time:", e)
```

```
# async_client_conversion_practice.py
from openai import AsyncOpenAI, APITimeoutError
```

```
import asyncio

client = AsyncOpenAI(timeout=30.0)


async def ask(question: str) -> str:
    response = await client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": question}],
    )
    return response.choices[0].message.content


async def main() -> None:
    try:
        answer = await ask("What is 2 + 2?")
        print(answer)
    except APITimeoutError:
        print("the API call took too long and was cancelled")
```

Fill in the `if __name__ == "__main__":` block yourself, then compare your finished versions against the Solution.

Two more async keywords, and LangChain's async methods:

- `async with AsyncOpenAI(timeout=30.0) as client:` — opens the client and closes its connections when the block ends (`client.is_closed()` is then `True`).
- `stream=True` , then `async for chunk in stream:` — a loop where each next chunk is awaited; print `chunk.choices[0].delta.content` when it isn't empty.
- On a Doc05 chain: `await chain.ainvoke({...})` , `async for piece in chain.astream({...}):` , and `await asyncio.gather(chain.ainvoke(...), chain.ainvoke(...))` to run several at once.

Difference between Basic and Intermediate: Basic creates `AsyncOpenAI()` with no timeout at all, trusting every call to come back quickly. Intermediate sets `timeout=30.0` once, at the client level, so every call made through that client is bounded by default — and catches the SDK's specific `APITimeoutError` instead of a bare `except Exception:` , so a genuine bug elsewhere in the function doesn't get silently mistaken for a slow API call.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution